

Session 4 · Deck 4.B4

Worked Example: The Stochastic RBC Model, Properly

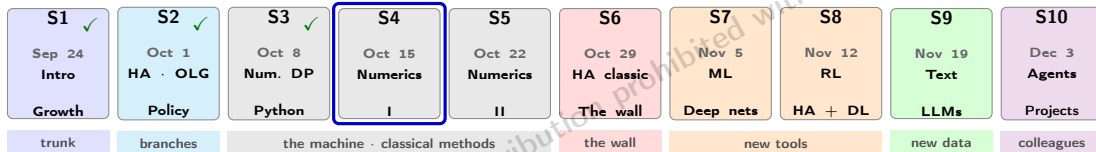
Zhigang Feng

Thursday, October 15, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 4.B4 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block B: 4.B1 approximation and interpolation · 4.B2 expectations: quadrature and the Markov chain · 4.B3 fast and accurate · 4.B4 the worked example, end to end.

The Same Deck as 3.A4, With the Tools Turned On

In 3.A4 every one of the six decisions was made the crudest way: a hand-written two-state chain, no interpolation, a grid search for the maximum, and a full sweep every iteration. We re-run *that solver* here — grid search and full sweeps, on this session's quarterly calibration and its 7-state chain — to get an honest “before” column.

Then the same five steps, with this session's replacements in place:

3.A4	here
II by hand	Rouwenhorst (4.B2)
no interpolation	linear , which keeps concavity (4.B1)
grid search	golden section (4.A1)
full sweeps	Howard , $n_H = 20$ (4.B3)

The model

$$v(k, z) = \max_{k'} \left\{ \ln c + \beta \mathbb{E}[v(k', z') \mid z] \right\}$$

$$c + k' = zk^\alpha + (1 - \delta)k, \quad \ln z' = \rho \ln z + \sigma \varepsilon'.$$

Quarterly: $\alpha = 0.33$, $\beta = 0.99$, $\delta = 0.025$,
 $\rho = 0.95$, $\sigma = 0.007$.

No closed-form policy. The benchmarks are the steady state, the chain's own moments, and the Euler residual.

Step 1: The Chain, and Checking It Before Anything Else

A seven-state chain for $\ln z' = 0.95 \ln z + 0.007 \varepsilon'$, built by **Rouwenhorst** (4.B2) — and checked before anything uses it.

- 1: $p \leftarrow (1 + \rho)/2$; $\Pi \leftarrow \begin{pmatrix} p & 1-p \\ 1-p & p \end{pmatrix}$
- 2: **for** $k = 3, \dots, 7$ **do** ▷ grow by one state
- 3: $\Pi \leftarrow p \begin{pmatrix} \Pi & 0 \\ 0 & 0 \end{pmatrix} + (1-p) \begin{pmatrix} 0 & \Pi \\ 0 & 0 \end{pmatrix} + (1-p) \begin{pmatrix} 0 & 0 \\ \Pi & 0 \end{pmatrix} + p \begin{pmatrix} 0 & 0 \\ 0 & \Pi \end{pmatrix}$
- 4: halve the interior rows
- 5: **end for**
- 6: $\sigma_z \leftarrow \sigma / \sqrt{1 - \rho^2}$; grid: 7 points on $\pm \sigma_z \sqrt{6}$; $z \leftarrow e^{\text{grid}}$
- 7: **check**: ρ and σ_z implied by Π against the targets

implied ρ	0.950000
implied σ_z	0.02241794
target σ_z	0.02241794
z range	[0.947, 1.056]

Check the chain before you solve anything.

Three lines, and it rules out the single most common silent error in this literature — a shock process that is not the one you wrote down.

Step 2: The Benchmark, and the Grid

The one number we know exactly, and a grid built around it.

- 1: $k^* \leftarrow ((1/\beta - 1 + \delta)/\alpha)^{1/(\alpha-1)}$ $\triangleright = 28.3484$
- 2: grid: $n_k = 300$ points, equally spaced on $[0.5k^*, 1.5k^*]$
 $\triangleright h = 0.0948$
- 3: cash on hand $y_{ij} \leftarrow z_j k_i^\alpha + (1 - \delta)k_i$ for every (k_i, z_j)

Cash on hand does not depend on v , so it is computed **once**, outside every loop — the same trick as 3.A2's payoff matrix.

$k^* = 28.3484$, $y^* = 3.0153$, $c^* = 2.3066$, and $K/Y = 9.40$ at a quarterly frequency — about 2.35 annual, which is the number to sanity-check against.

The grid is $\pm 50\%$ around k^* . Print $\max_i g(k_i)/k_N$ when you are done: if it is 1, the policy is pinned at the top of the grid and the answer is censored, not solved.

Step 3: The Inner Maximization Is No Longer a Search

For each state, maximize the Bellman right-hand side over k' *on the interval*, not over the grid — one bracket per state, all states at once.

- 1: $R(k') \equiv \ln \max(y - k', 10^{-12}) + \beta \widehat{EV}(k')$ $\triangleright \widehat{EV}$: linear interpolation
- 2: bracket $[a, b] \leftarrow [k_1, \min(y - 10^{-8}, k_N)]$
- 3: $c \leftarrow b - \varphi(b - a)$, $d \leftarrow a + \varphi(b - a)$; evaluate $R(c), R(d)$
- 4: **for** 40 times **do**
- 5: **if** $R(c) > R(d)$ **then** keep $[a, d]$; old c becomes the new d
- 6: **else** keep $[c, b]$; old d becomes the new c
- 7: **end if**
- 8: evaluate R at the **one** new interior point
- 9: **end for**
- 10: $g(k, z_j) \leftarrow (a + b)/2$

Three of this session's pieces, in one block:

- $\widehat{EV}$ is **4.B1's** linear interpolation — chosen because it keeps $\hat{v}$ concave;
- that concavity is what makes the objective **unimodal**;
- which is what licenses **4.A1's** golden section to shrink the bracket safely.

Swap in a cubic spline and the first bullet fails, so the third one does too. **The interpolation choice is not cosmetic** — it is a hypothesis of the maximizer.

Step 4: The Expectation, and Howard Around the Outside

The expectation once per sweep; then Howard steps that reuse the policy.

```
1:  $V \leftarrow 0$ 
2: repeat
3:    $EV \leftarrow V \Pi^\top$   $\triangleright \mathbb{E}[v(k', z') \mid z], \text{ one product}$ 
4:   for each  $z_j$ :  $g(\cdot, z_j) \leftarrow$  golden section (Step 3);
      $V^{\text{new}}(\cdot, z_j) \leftarrow R(g)$ 
5:    $d \leftarrow \max |V^{\text{new}} - V|$ ;  $V \leftarrow V^{\text{new}}$   $\triangleright$  copy, not rebind
6:   for 20 times do  $\triangleright$  Howard:  $g$  held fixed
7:      $EV \leftarrow V \Pi^\top$ ; for each  $z_j$ :  $V(\cdot, z_j) \leftarrow R(g(\cdot, z_j))$ 
8:   end for
9: until  $d < 10^{-6}$ 
```

The product $V \Pi^\top$ is the whole expectation — $\mathbb{E}[v(k', z') \mid z]$ for every (k', z) at once. It is computed **once per sweep**, outside the maximization, not re-summed inside it.

The inner loop is Howard: the policy g is *held fixed* and only the value is updated, so there is no maximization and no bracket — just an interpolation and an add.

Two nested loops, and they do completely different work. Confusing them is the most common way this algorithm is implemented wrongly.

Step 5: What It Bought, Stage by Stage

Same model, same grid ($n_k = 300$), same chain, same tolerance — only the inner maximization and the sweep structure differ.

	3.A4 style	this deck
policy updates	1,352	66
wall time	1.04 s	0.69 s
inner work	852M comparisons	5.8M evals
worst $\log_{10} \mathcal{E}$	-1.30	-2.52
mean $\log_{10} \mathcal{E}$	-2.14	-3.40
distinct k' / 36 states	35	36

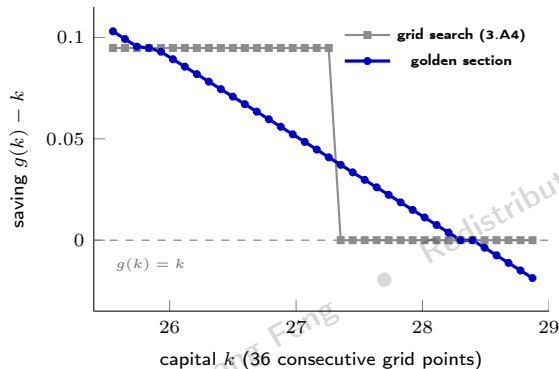
1.2 orders of magnitude of accuracy, a twentieth of the policy updates, and inner work down by a factor of 150 — at two-thirds of the wall clock. 4.B3 runs the same comparison at $n_k = 500$ with a Tauchen chain (-1.55 and -2.77); the gain there is also 1.2 orders of magnitude.

Where each gain came from

The **iteration count** is Howard. The **accuracy** is golden section plus interpolation. The **chain** changed nothing about speed and everything about what the model is.

But do not read too much into the clock. A $150\times$ drop in inner work buys only a third off the time, because golden section runs in a Python loop while the grid search is one array operation. Report the operation count, which is implementation-independent, *and* the clock.

The Staircase, Retired

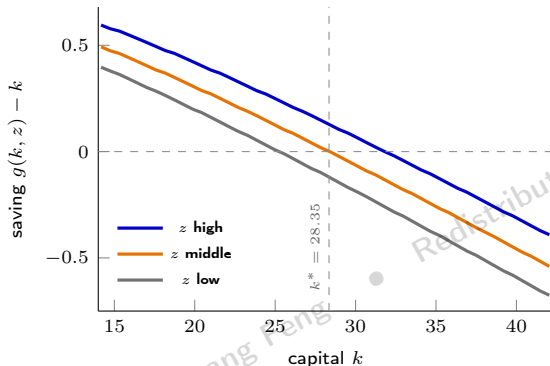


Thirty-six consecutive grid points in the middle state, plotted as *saving* rather than k' — the same data, with the 45-degree line subtracted off.

- Grid search can only pick $k' = k + mh$, so saving takes **two values in this whole window**: one grid spacing, then zero. It is a staircase with two steps.
- Golden section gives a **smooth, declining** line that crosses zero once.

And the two cross zero a full **ten grid spacings apart** — 27.35 against $k^* = 28.35$. The plot of k' against k hides this entirely; subtract the 45-degree line and it is unmissable.

The Policy, and the Three Checks



Three things to look at, in this order:

1. The three curves are **ordered in z** and do not touch — a good shock saves more at every k . A reversed z grid breaks this immediately.
2. Saving is **declining in k** — $g'(k) < 1$ — which is what makes the steady state stable.
3. The middle curve crosses zero at $k^* = 28.35$; the low and high curves cross on either side. Those are the model's conditional steady states.

Note how *shallow* these curves are — saving moves by about 1 unit while k moves by 28. That is why the previous slide's grid-search crossing missed by ten spacings: a flat function makes a weak test. **Use the residual to measure accuracy**; use the picture for sign checks only.

What Is Still Crude

Even with the whole Session 4 toolkit switched on, three things in this code are still the cheap option:

- **Linear interpolation.** $O(h^2)$, and the kinked derivative is felt by anything that differentiates $\hat{v}$. A shape-preserving spline would do better without losing concavity.
- **One continuous state.** Add a second asset and the grid is n_k^2 — and **4.B1's Smolyak** is the first answer, S5's projection the second.
- **We still iterate on the value function.** **4.B3's EGM** skips it entirely: at $n_k = 500$ it reaches $10^{-6.06}$ against $10^{-2.77}$ for this method — **three further orders of magnitude**, and ten times faster.

The honest summary

This is a good solver. It is not the best one in the session — EGM is — and the reason to write this one first is that **every piece of it generalises** to models where EGM does not apply.

EGM needs an invertible u' , one continuous control and a monotone policy. Value iteration needs none of those; golden section needs only a unimodal objective — and with a discrete choice on top, VFI over the discrete options is still the workhorse.

How It Works: Steps 1 and 2 in Python

Step 1: the chain

```
def rouwenhorst(n, rho, sigma):  
    p = (1 + rho) / 2  
    P = np.array([[p, 1-p], [1-p, p]])  
    for k in range(3, n+1):  
        Z = np.zeros((k, k))  
        Z[:-1,:-1] += p*P;    Z[:-1,1:] += (1-p)*P  
        Z[1:,:-1] += (1-p)*P; Z[1:, 1:] += p*P  
        Z[1:-1, :] /= 2.0  
        P = Z  
    sz = sigma / np.sqrt(1 - rho**2)  
    psi = sz * np.sqrt(n - 1)  
    return np.linspace(-psi, psi, n), P  
  
logz, Pi = rouwenhorst(7, 0.95, 0.007)  
z = np.exp(logz)
```

The full solver, runnable, is Lab L04, Part 7.

Step 2: the benchmark and the grid

```
alpha, beta, delta = 0.33, 0.99, 0.025  
  
# the one thing we know exactly  
kss = (((1/beta - 1 + delta)/alpha)**(1/(alpha-1)))  
# 28.348419061048443  
  
n_k = 300  
kg = np.linspace(0.5*kss, 1.5*kss, n_k)  
h = kg[1] - kg[0] # 0.0948  
  
# cash on hand, every (state, shock)  
y = (z[None,:]*kg[:,None]**alpha  
     + (1-delta)*kg[:,None])
```

How It Works: Steps 3 and 4 in Python

Step 3: golden section, vectorised over k

```
PHI = (np.sqrt(5) - 1) / 2
```

```
def rhs(kp, yj, EVj):    # Bellman RHS, off grid
    c = np.maximum(yj - kp, 1e-12)    # clamp
    return np.log(c) + beta*np.interp(kp,kg,EVj)
```

```
# golden section, vectorised over all k at once
```

```
yj, EVj = y[:, j], EV[:, j]
a = np.full(n_k, kg[0])
b = np.minimum(yj - 1e-8, kg[-1])
c_, d_ = b - PHI*(b-a), a + PHI*(b-a)
fc, fd = rhs(c_, yj, EVj), rhs(d_, yj, EVj)
W = np.where
for _ in range(40):
    L = fc > fd    # keep [a,d] or [c,b]
    b, a = W(L, d_, b), W(L, a, c_)
    x = W(L, b - PHI*(b-a), a + PHI*(b-a))
    fx = rhs(x, yj, EVj)    # ONE new evaluation
    c_, d_ = W(L, x, d_), W(L, c_, x)
    fc, fd = W(L, fx, fd), W(L, fc, fx)
g[:, j] = 0.5*(a + b)
```

Step 4: expectation and Howard

```
V = np.zeros((n_k, n_z)); Vn = np.empty_like(V)
while True:
    EV = V @ Pi.T    # (n_k,n_z): one matmul
    for j in range(n_z):
        g[:, j] = golden_section(j, EV)
        Vn[:, j] = rhs(g[:, j], y[:, j], EV[:, j])
    d = np.max(np.abs(Vn - V))
    V[:, :] = Vn    # copy, do not rebind

    for _ in range(20):    # Howard: no argmax
        EV = V @ Pi.T
        for j in range(n_z):
            V[:,j] = rhs(g[:,j], y[:,j], EV[:,j])
    if d < 1e-6:
        break
```

Takeaways

- The same five steps as 3.A4, with four substitutions: **Rouwenhorst, interpolation, golden section, Howard**. The algorithm is barely longer.
- **Check the chain first**. Implied $\rho = 0.950000$ and σ_z matching the target to eight digits — three lines that rule out the commonest silent error in the literature.
- The interpolation choice is a **hypothesis of the maximizer**: linear keeps $\hat{v}$ concave, concavity gives unimodality, and unimodality is what licenses golden section. A cubic spline breaks the chain at the first link.
- Measured: **66 policy updates against 1,352**, inner work down $150\times$, and **1.2 orders of magnitude** of Euler accuracy. The wall clock falls by only a third — which is why you report the operation count too.
- It is still not the best solver here. **EGM is**, by three further orders of magnitude — but this one needs no invertible u' and survives into models with discrete choices, where plain EGM cannot go.

Bridge: Two Sessions, One Model, Many Solvers

Sessions 3 and 4 have now solved the same model many different ways — grid VFI, time iteration, Howard, monotonicity, continuous choice, EGM — and every difference between them was measured rather than asserted. That is the habit the rest of the course rests on.

What has not changed is the **representation**: values, on a grid, one number per state. Everything we have built makes that representation cheaper or more accurate. None of it makes the grid go away — and at two continuous states it already hurts.

Next week (S5)

Perturbation discards the grid and expands around the steady state. **Projection** replaces N separate values with a handful of basis coefficients — and Smolyak returns, now as the basis rather than the node set.

Then this same model is solved both ways, and put beside today's.

Homework 1 is due October 29. Part B solves this deck's model, and B1(b) asks you to build the chain both ways and see whether it moved the answer.